

RESULTS

Phase-3 CPU OCR (Tesseract) — end-to-end proof (§11.4.108 / §11.4.5 / §11.4.69)

Date: 2026-07-07 · Track (T1/feature/helixllm-full-extension) · One of 3 parallel Phase-3 extended-capability streams (docs/research/07.2026/00_master/RESUME.md): translation (:18436), Whisper STT (:18437), **Tesseract OCR (:18438, this doc).**

Verdict (honest, §11.4.6)

CAPABILITY PROVEN. A real CPU OCR service — Tesseract 5 (OEM 1 / LSTM) behind a minimal Go HTTP shim (submodules/helix_llm/services/ocr/) — was booted via the **containers submodule orchestrator** (§11.4.76, rootless podman §11.4.161, NO GPU), rendered two independent KNOWN-text fixtures **inside its own container** (so no host font/version could ever fake the result), OCR'd them, and the extracted text matched the known ground truth at high confidence. The §11.4.108 runtime signature is **GREEN-OK** for both strings:

```
[RUNTIME-SIGNATURE] PASS mean_conf=95.99 found=[HELIX OCR 2026 QUICK BROWN
                    full_text="HELIX OCR 2026 quick brown fox"
[RUNTIME-SIGNATURE] PASS mean_conf=94.77 found=[PHASE THREE TESSERACT PROO
                    full_text="PHASE THREE TESSERACT PROOF SEVEN"
[DETERMINISM]       PASS identical full_text/mean_conf/word-count across t
[SELF-VALIDATION]   PASS: analyzer PASSes golden-good and FAILs all golden
```

Unfakeable by construction: the harness rendered the known text itself (via the service's own `/v1/render`, from a raw string literal it controls), then recovered it via a *separate* `/v1/ocr` call — the only way this passes is if Tesseract genuinely read pixels it was never told the answer to.

Runtime-signature block + confidence-floor rationale (§11.4.6 / §11.4.107(13))

CONF_FLOOR=60, calibrated **from this run's own observed data**, never from literature:

Fixture	mean_conf observed	Expected-token check
golden-good #1 “HELIX OCR 2026 quick brown fox”	95.99	all 6 tokens found
golden-good #2 “PHASE THREE TESSERACT PROOF SEVEN”	94.77	all 5 tokens found
golden-bad blank (pure white, no text)	0.00	0/6 found
golden-bad noise (+noise Random canvas)	10.58 – 11.97 (varies per random draw)	0/6 found
golden-bad wrong- content (“BANANA SPACESHIP GALAXY EIGHT”)	95.98	0/6 found

60 sits comfortably below the observed good cluster (~95-96) and decisively above the highest observed bad-fixture confidence (~12) — the same separation the pre-existing local prototype (`docs/qa/p3_tesseract_ocr/README.md`, a prior, non-compliant ad-hoc `podman run/podman build` exploration, superseded by this deliverable) found with its own floor of 80 on the same class of rendered label text (94-96 good cluster) — independent cross-validation that this confidence range is a real, repeatable Tesseract behaviour on clean rendered UI-style text, not a fluke of this run.

The wrong-content fixture is the load-bearing anti-bluff case: it scores a *high* mean_conf (95.98, correctly-recognized real text) yet still **FAILs**, because confidence alone cannot detect “the right pixels, wrong content.” Only the **expected-token-set** check catches it — proving the analyzer needs both criteria, not confidence alone (`13_self_validation.txt`).

Analyzer is non-bluff (§11.4.107(10) / §11.4.115)

- **RED baseline** (`10_red_baseline.txt`): a stub response (`full_text: ""`, `mean_conf: 0`, `words: []` — the shape a dead/broken OCR engine would return) was fed to the analyzer with `RED_MODE=1` semantics and correctly **FAILED** (exit 1) *before* the real container was even booted — defect-shape reproduced first.

- **Golden-BAD fixtures** all correctly **FAIL** (13_self_validation.txt): blank (zero recognized text), noise (mean_conf ~11, zero token overlap despite Tesseract hallucinating a few low-confidence glyphs from random pixels), wrong-content (high confidence but zero token overlap).
- **Real GREEN input** correctly **PASSes**, for *two independently rendered strings* (not a one-image fluke).

Honest substitution (§11.4.6)

The Containerfile was originally two-stage: a golang:1.25-bookworm build stage compiling ocr-shim, copied into a debian:bookworm-slim runtime stage. That build **reproducibly FAILED** on this host — podman-compose build aborted unpacking the golang base image's layers with:

```
runtime/cgo: pthread_create failed: Resource temporarily unavailable
SIGABRT: abort
```

and, on the very next attempt (smaller debian:bookworm-slim-only pull), the apt-get install RUN step *itself* failed the same way:

```
error running container: reading container state from /usr/bin/crun (got o
fork/exec /usr/bin/crun: resource temporarily unavailable
```

Root-caused (§11.4.102), not guessed: ulimit -u (this UID's soft RLIMIT_NPROC) is **4096**; pids.current for this session's cgroup was **3904** (~95% consumed), dominated by many long-running, unrelated node .../mongodb-mcp-server processes (75 threads each) accumulated from other sessions on this shared host — **not** anything this build did, and not a process this task is authorized to inspect ownership of or kill (§11.4.174 — a live MCP server in another session is out of scope for autonomous action). The fix, self-scoped only: 1. ulimit -u "\$(ulimit -Hu)" at the top of run_proof.sh — raises **only this script's own process tree's** soft limit to its hard cap (5120), touching no other process. 2. The Containerfile was changed to **single-stage**: ocr-shim is now compiled *outside* the container (CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build,static,by run_proof.sh itself before every boot — a §11.4.77 regeneration mechanism, see submodules/helix_llm/services/ocr/README.md “Build the shim binary”) and simply **COPYed** in, eliminating the heavyweight golang-toolchain image pull/unpack entirely. This is a substantially lighter build (one small Debian base + five apt packages) that succeeded where the two-stage build did not — evidence in 20_boot.txt (UP-OK) and 24_container_state.txt.

Both changes are captured in the Containerfile's own comments and submodules/helix_llm/services/ocr/README.md. Nothing was faked to route around the failure; the exact errors are quoted above verbatim from the actual failed runs.

§11.4.76 / §11.4.161 / §11.4.119 compliance

- Booted via `digital.vasic.containers/pkg/compose.Orchestrator` (`Up(..., WithBuildFirst(true))`) → the orchestrator's detected `compose` command runs the build, never an ad-hoc `podman build/podman run`. Rootless `podman` throughout.
- Host port **18438** — distinct from `coder :18434`, `embeddings :18435`, `translation :18436`, `Whisper :18437` (verified free before boot, `00_preflight.txt`).
- Unique per-run `compose` project name `phase3ocr`; pre-clean boot-down before boot for clean-target integrity (§11.4.108/§11.4.139).
- Single-owner teardown (`29_tearardown.txt`): `DOWN-OK`, container removed (`29b_post_tearardown.txt`). `helixllm-coder` confirmed **Up 21 hours** before, during, and after this run — never touched. No sibling Phase-3 capability containers were present or affected (`00_preflight.txt`, `29b_post_tearardown.txt`).

Reproduce

```
cd docs/qa/phase3_tesseract_ocr_20260707/harness
./run_proof.sh
```

Builds the harness + the `ocr-shim` static binary, boots the container via the `containers` submodule `orchestrator` (`--build`), renders + OCRs both known-text fixtures plus the three golden-bad fixtures, asserts the runtime signature + determinism + self-validation, tears the container down single-owner, and leaves `helixllm-coder` untouched. Config knobs (`env`, all default-injected — §CONST-045/046): `OCR_HOST_PORT` (18438), `OCR_MEM_LIMIT` (2g), `OCR_CPUS` (4), `HEALTH_TIMEOUT` (300s), `CONF_FLOOR` (60). `harness/phase3ocr.bin` and `submodules/helix_llm/services/ocr/ocr-shim` are gitignored build artifacts (§11.4.30), regenerated automatically by `run_proof.sh`.

Deep multi-angle research (§11.4.150) — sources verified 2026-07-07

1. **Debian package tracker** — `tesseract-ocr` in `bookworm` resolves to **5.3.0-2** (Tesseract 5.x line; OEM 1/LSTM is the compiled-in default) — <https://packages.debian.org/bookworm/tesseract-ocr> (accessed 2026-07-07). No backports repo needed for OEM-1/LSTM behaviour; only the *patch* version (5.3 vs upstream-latest 5.5.x) differs, which does not affect the `tsv/image_to_data` contract this service relies on.
2. **tessdoc Command-Line-Usage** — confirms the exact CLI invocation (`tesseract <image> <outputbase> --oem 1 --psm <n> tsv, - as`

outputbase → stdout) and the TSV column layout (level page_num block_num par_num line_num word_num left top width height conf text) — <https://tesseract-ocr.github.io/tessdoc/Command-Line-Usage.html> (accessed 2026-07-07).

3. **TSV conf column semantics** — word-level (level==5) rows carry conf 0-100; all other levels carry conf=-1 (layout-only, no confidence) — cross-confirmed by <https://tomrochette.com/tesseract-tsv-format/> and <https://github.com/tesseract-ocr/tesseract/issues/2746> (both accessed 2026-07-07). This is exactly the parsing rule implemented in submodules/helix_llm/services/ocr/main.go's parseTSV.
4. **Prior local empirical corroboration** (this repo, docs/qa/p3_tesseract_ocr/, dated shortly before this run) — an earlier, non-compliant (ad-hoc podman build/podman run, not the containers-submodule orchestrator) exploration of the *same* Tesseract-in-container approach independently observed mean confidence **94-96** on a similarly-rendered “HELIX OCR TEST 42” label and confirmed Debian tesseract-ocr resolves to **5.3.0**, not the upstream-latest 5.5.x — cross-validating both this run's confidence cluster and the package-version finding from angle 1. This directory is prior art / research corroboration only; it is **not** part of this deliverable (different boot mechanism, different port, superseded).
5. **Confirm-no-bigger-problem (§11.4.150(C))**: the two independent failures during this run (golang-image unpack, then plain apt-get install) both carried the identical resource temporarily unavailable / pthread_create failed signature, which — cross-checked against ulimit -u / cgroup pids.current on this host (§11.4.102 investigation above) — confirms a single root cause (host-wide per-UID process-count pressure from unrelated long-lived processes), not two unrelated defects and not a defect in the OCR service's own design.

No external solution was needed beyond the above — the fix (self-scoped ulimit bump + single-stage build) is original engineering for this specific host constraint, cited here per §11.4.8's “NO external solution found for the exact remediation — original work” clause (the *facts* about Tesseract/Debian/TSV that informed the design are all externally sourced above).

Composition

§11.4.76 (containers submodule) · §11.4.161 (rootless) · §11.4.108 (runtime signature) · §11.4.107(10) (self-validated analyzer) · §11.4.115 (RED-first) · §11.4.119 (single-owner teardown) · §11.4.174 (process-ownership verified — no other process touched) · §11.4.102 (systematic-debugging root-cause before fix) · §11.4.6 (honest substitution + honest limitation) · §11.4.150 (deep multi-angle research, cited above).

Honest boundary (§11.4.6)

This proves a **real CPU OCR service** genuinely reading known text back out of rendered pixels it was never told, with a non-bluffable golden-bad battery (blank / noise / wrong-content) and a RED-first empty-stub baseline. It does **not** prove multi-language OCR, layout/table extraction, handwriting, the PaddleOCR/Surya ML-fallback tier, or the VLM last-resort tier described in the broader design doc (docs/research/07.2026/07_stt_ocr_whisper_tesseract/07_stt_ocr_whisper_tesseract.md §2.3) — only the Tier-1 Tesseract default lane, end-to-end, containerized, non-bluff.